Anchored Semantic Reduction: Locating and Policing the Bottleneck of Reliable AI for Mathematics

*Author**

September 28, 2026

Abstract

AI systems now produce mathematical arguments that are fluent, plausible, and wrong. The dominant response is to add *verification*: search-and-check loops, formal proof assistants, and reproducible evaluation. We argue that this response is aimed one layer too late. Decomposing an AI-for-mathematics (AI4Math) pipeline into *generation*, *semantic reduction*, and *verification*, we show that the reliability of the whole pipeline is normally governed by the middle layer — the translation from natural language into the object the verifier actually consumes — and that this layer is routinely left unmonitored. We give two propositions. The first is an *existence* result: because a sound-and-complete verifier can only be sound *relative to the object it is handed*, an unfaithful reduction yields a certified, machine-checked, wrong answer. The second is a *structural* impossibility: any verifier that sees only the reduced object is a function of that object, so it cannot separate two sentences the reduction conflates — its detection recall on such pairs is zero, not merely low. We then build an exactly decidable experimental harness over a small typed intermediate representation of finite-domain affine arithmetic, generate a controlled corpus of semantics-changing perturbations across five failure classes, and compare two ways of *anchoring* the reduction: comparing against a re-rendering of the intermediate object (A-lex), and asserting independently extracted statements about meaning as predicates over that object (A-type). On a prevalence-controlled corpus, plain verification reports a 55.9% silent error rate while certifying 100% of its answers; typed anchors drive silent error to 0%. A further ablation at the canonical surface form shows that A-lex's headline detection recall is partly a paraphrase artefact — it collapses to 0% on typing drift — whereas A-type reaches 100% recall with no certification cost. We close with an eight-item reliability checklist and an explicit statement of the conditions under which anchoring cannot help.

1 Introduction

An AI system asked to settle a mathematical question will, today, usually give an answer that reads as if it had been reasoned out. Sometimes it has been. The problem that has organised the field is that the two cases are hard to tell apart: a model can produce a derivation whose every step looks locally reasonable, which reaches a conclusion that is false, and which no one catches because the derivation is only ever read, never checked [Yang et al., 2024]. The failures are not noise. They are systematic, they survive scale, and they are most likely precisely where the problem is hard.

*Author block intentionally left generic. Replace before submission. The experimental code, the generated tables and figures, the AI collaboration log and the after-action review accompany this paper.

The community's main answer has been *verification*. Three strands are visible. The first pairs a generative model with a search-and-check loop, so that candidate steps are evaluated before being extended [Lightman et al., 2024, Uesato et al., 2022]. The second replaces natural-language argument with formal proof, so that a machine checks every inference and a proof cannot be subtly wrong [de Moura and Ullrich, 2021, Hubert et al., 2025, Trinh et al., 2024]; the translation step that carries a problem from prose into the formal system is autoformalization [Wu et al., 2022, **?**]. The third makes evaluation reproducible and comparable across systems [Zheng et al., 2022, Romera-Paredes et al., 2024, Charton et al., 2024]. Each strand has produced genuine, durable progress.

This paper makes a claim about *where* that progress has to be applied, and about what happens when it is applied everywhere except there. Consider a pipeline in the abstract:

$$\underbrace{\text{statement}}_{\text{natural language}} \longrightarrow \underbrace{\text{object}}_{\text{what the checker eats}} \longrightarrow \underbrace{\text{verdict, certificate}}_{\text{checked}}$$

The middle arrow is a *semantic reduction*. It is where the meaning of an informal sentence is fixed. It is also, in most systems, a preprocessing detail: the model's reading of the problem is passed straight to the checker, and the checker is trusted because it is sound. We argue that this is a category error. A checker is sound *relative to the object it is handed*. If the object is not what the sentence meant, soundness buys nothing at all, and the resulting answer is not merely unverified — it is verified and wrong, which is strictly worse than an answer that was obviously a guess.

The contribution of this paper is to make that argument precise, then to test it.

Contributions.

1. **A three-layer error model** (§3) that separates generation, semantic reduction, and verification, and localises reliability in the middle layer. We argue that most reported AI4Math reliability failures are reduction failures wearing a verification costume.

2. **A failure taxonomy of the reduction layer** (§3.3): quantifier drift, domain-kind (typing) drift, strictness drift, assumption omission, and vacuity. The taxonomy is instantiated and measured, not merely proposed.

3. **Two propositions** (§3.4). Proposition 1 shows that certified wrongness is reachable even when the verifier is exactly sound and complete. Proposition 2 shows that an IR-only verifier has detection recall *identically zero* on pairs the reduction collapses — a structural statement, not an empirical accident.

4. **An exactly decidable experimental harness** (§4). The intermediate representation is a typed, finite-domain affine fragment decided by exact enumeration over rationals, so no floating-point tolerance can mask an error, and the whole study is deterministic given a seed.

5. **A comparative result on anchoring** (§5). Lexical round-trip anchors (A-lex) and typed predicated anchors (A-type) are measured as reduction-error detectors. We show which of them is honest and why, including an ablation at the canonical surface form that removes a paraphrase confound from A-lex.

6. **An eight-item reliability checklist** (§3.6) and a statement of the conditions under which anchoring *cannot* help (§6.2).

We are explicit about what we do *not* claim. We do not claim that formal verification is unnecessary; we claim it is insufficient on its own. We do not claim that our controlled language is natural language; it is a deliberately small, fully specified fragment whose semantics are exact, which is what makes the measurements trustworthy. And we do not claim that anchors work unconditionally: §6.2 states the independence condition they require, and what happens when it fails.

2 Related Work

Step-level supervision and verification of informal reasoning. Uesato et al. [2022] ran the first systematic comparison of outcome-based against process-based supervision on a natural-language reasoning task, finding that outcome-only supervision reaches similar final-answer accuracy with fewer labels while failing to reduce reasoning errors among correct answers. Lightman et al. [2024] scaled the process-supervised reward model to the MATH dataset with 800,000 step-level human labels and showed a clear advantage for process supervision under best-of-N reranking (78.2% against 72.4% for an outcome-supervised model). This line of work is, in our terms, about *which step to believe*. It improves the generation layer and adds a checking signal over steps. It does not, however, monitor whether the problem was read correctly in the first place — a process reward model scores a *given* chain of reasoning, and a uniformly well-scored chain restating the wrong question is still wrong.

Formal proof and autoformalization. Formal systems remove the bottom of the error stack: a proof that type-checks cannot be subtly invalid [de Moura and Ullrich, 2021]. This has been turned into systems that reach olympiad-level performance within the formal environment [Trinh et al., 2024, Hubert et al., 2025]. But the formal environment must be entered. Autoformalization is the bridge: Wu et al. [2022] showed that large language models translate a substantial fraction of competition problems correctly into Isabelle/HOL — and, importantly, reported that a significant fraction is *not* translated correctly, with the failures being of exactly the character our taxonomy describes. Jiang et al. [2023] route informal proofs through formal sketches to guide a prover, raising the solve rate from 20.9% to 39.3%, again with the informal-to-formal mapping as the critical, and lightly supervised, component.

Our contribution relative to this literature is not a better translator. It is a statement about what a translator must be paired with in order for the pipeline to be trustworthy at all, together with an empirical demonstration that the obvious pairing — the round-trip — is not it.

Program search, constructions and counterexamples. Romera-Paredes et al. [2024] pair a language model with a systematic evaluator and search in program space, obtaining new constructions for the cap set problem; Charton et al. [2024] alternate local search with a transformer to find extremal constructions, including a counterexample to a conjecture open for thirty years. These systems are protected by an evaluator that is exact for the object being searched. They are the positive control for our argument: reliability comes from having *some* layer whose semantics are fixed and checkable, and from keeping the generative layer away from it.

Surveys and positioning. Yang et al. [2024] argue that formal mathematical reasoning is the avenue most likely to carry AI4Math beyond its current ceiling, and identify autoformalization as a central open problem. Our paper takes that assessment seriously and sharpens it: the open problem is not only *how to translate better*, but *how to know when a translation is wrong*, which

is a monitoring problem rather than a modelling problem, and admits a structural impossibility result.

3 Framework: Anchored Semantic Reduction

3.1 Three layers, and where the errors enter

We decompose an AI4Math pipeline into three layers.

Definition 1 (Three-layer pipeline)**.** A pipeline consists of

1. a *generation* map G that produces a natural-language answer, argument, or question from a problem statement s;
2. a *semantic reduction* R that maps a natural-language statement to an intermediate representation (IR) $S = R(s)$;
3. a *verification* map V that maps an IR to a verdict and, ideally, a machine-checkable certificate.

The pipeline's *answer* on s is $V(R(s))$. Write $[\![S]\!]$ for the truth value the IR S denotes, and ρ^* for the *faithful* reduction, i.e. the one satisfying $[\![\rho^*(s)]\!] = \mathrm{truth}(s)$ for every sentence s in the domain of interest. Then R is *faithful on* s iff $R(s) = \rho^*(s)$, and *unfaithful on* s otherwise.

Note the asymmetry in Definition 1. Faithfulness is a property of R; soundness is a property of V. The literature's investment is concentrated in V. The next subsection shows why that is not enough.

3.2 A typed IR over a decidable fragment

For the experiment we need an IR whose semantics are exact and whose verdicts are reproducible to the bit. We use a small typed fragment.

Definition 2 (IR)**.** A specification S is a tuple $(\mathcal{V}, H, g, Q)$ where

- $\mathcal{V} = \{(v_i, \ell_i, u_i)\}$ declares each variable v_i as an integer constrained to a finite closed interval $[\ell_i, u_i]$;
- $H = \{h_1, \ldots, h_p\}$ is a finite set of hypotheses, each a linear constraint $\sum_j c_j v_j \bowtie k$ with rational coefficients c_j, rational k, and $\bowtie \in \{<, \leq, =, \geq, >\}$;
- g is a goal, of the same form;
- $Q \in \{\mathsf{EXISTS}, \mathsf{FORALL}\}$ is the *quantifier of the question*: is g satisfiable together with H, or does H entail g?

The quantifier is part of the IR deliberately: dropping it is one of the failure modes we study. Decisions are computed by exact enumeration of the declared box using rational arithmetic (`fractions.Fraction` in Python), so a verdict, and the ground truth it is compared against, are exact integers or exact rationals throughout — there is no tolerance to tune and none to hide behind. The fragment is decidable and the procedure is sound *and* complete inside it, which is the point: we want a verifier that cannot be blamed.

3.3 Failure taxonomy of the reduction layer

The following five classes are the reduction errors our corpus instantiates. Each is a distinct way for two sentences with different truth values to collapse, or for a claim to become true for the wrong reason.

Definition 3 (Reduction failure classes)**.** Let s be a statement and $R(s)$ its IR.

P1 Quantifier drift. s asks a universal question; $R(s)$ encodes an existential one (or conversely). "For every x" becomes "for some x".

P2 Domain-kind (typing) drift. s restricts the domain *by kind* ("positive", "non-negative", "arbitrary"); $R(s)$ records only an interval and loses the kind, so a boundary witness the sentence excluded becomes admissible.

P3 Strictness drift. s uses a strict inequality; $R(s)$ weakens it (or conversely), which changes the answer exactly at the extreme the comparison pins down.

P4 Assumption omission. s carries a qualifying clause ("and x is a multiple of m"); $R(s)$ drops it, so a hypothesis that removes the only witness disappears.

P5 Vacuity. s is a universal claim whose hypotheses admit no model in the declared box. It is therefore *true*, but true of nothing; a pipeline that reports "true" without the warning has answered a different question from the one asked.

P1–P4 are semantics-changing: the truth value flips. P5 is subtler — the truth value does not flip, but the *informativeness* does, and P5 is the class most likely to survive every accuracy-based check.

3.4 Two propositions

Proposition 1 (Certified wrongness)**.** *Let V be sound and complete for the IR fragment, so that $V(S) = [\![S]\!]$ for every well-formed S. If R is unfaithful on s* and $[\![R(s)]\!] \neq \mathrm{truth}(s)$*, then the pipeline emits a verdict that carries a machine-checkable certificate and is wrong.*

Proof. The pipeline answers $V(R(s))$. Faithfulness of V gives $V(R(s)) = [\![R(s)]\!]$. By hypothesis $[\![R(s)]\!] \neq \mathrm{truth}(s)$, hence the emitted verdict differs from the truth. The verdict remains accompanied by V's certificate, which is internally valid: it witnesses the verdict for the *wrong* object. □ □

Proposition 1 is why the combination of high accuracy and high certificate rate is not reassuring on its own, and why we report silent error rate — wrong answers *emitted as certified* — separately from accuracy. The proof is one line; the consequence is not obvious in practice, because a pipeline that certifies everything looks like a pipeline that has been checked.

Proposition 2 (Structural blind spot of IR-only verification)**.** *Let V be any verifier whose input is the IR alone, so V is a function of S. Let s, s' be two statements with $R(s) = R(s')$ but* $\mathrm{truth}(s) \neq \mathrm{truth}(s')$*. Then*

1. *the verdicts agree, $V(R(s)) = V(R(s'))$, and so do the certificates;*
2. *V is wrong on at least one of s, s';*
3. *consequently, on the pair (s, s'), V's detection recall is* 0*.*

Proof. (1) Since V is a function and $R(s) = R(s')$, we have $V(R(s)) = V(R(s'))$; the certificate is part of V's output, so it coincides too. (2) A single verdict cannot equal two different truth values, so it differs from truth$(\cdot)$ on at least one member. (3) Detection requires the verifier to signal a difference between the two cases; by (1) its output is identical, so it signals nothing. □ □

Corollary 1. *Replacing V by a stronger or smarter IR-only verifier cannot raise recall above* 0 *on the class of pairs collapsed by R. Detection of reduction error is therefore not obtainable by improving the verifier; it requires a source of information that does not pass through R.*

Corollary 1 is the design constraint that motivates anchors. The zero row in Table 3 for "IR verification alone" is a consequence of Proposition 2, and we verify empirically that 100% of our critical pairs are collapsed in exactly this way (§5.1).

3.5 Anchors: lexical round-trip versus typed predicates

An *anchor* is a statement about what a problem means, extracted from the sentence by a route that does not pass through R. Anchors are what Corollary 1 demands. We compare two ways of turning an anchor into a check.

Definition 4 (A-lex, lexical round-trip)**.** Render $R(s)$ back to a sentence $\hat{s}$ using a partial inverse of the reductor, then compare s and $\hat{s}$ slot by slot, by exact surface equality. Report a violation when a slot differs. *Semantically irrelevant* differences — synonyms — are indistinguishable from relevant ones.

Definition 5 (A-type, typed predicated anchors)**.** Extract the anchors from s with a separately declared grammar, then assert them as *predicates* over $R(s)$: the quantifier field, the declared domain as a set, the *solution set* of the goal over that domain, and the presence and modulus of a congruence witness. Report a violation when a predicate is contradicted.

The difference is not cosmetic, and it is sharpest on P2. The IR has no slot for domain *kind*: it records only the resulting interval. A round-trip therefore reproduces the interval faithfully and annihilates the type information, so A-lex cannot see typing drift at all. A-type can, because it carries the intended typing rule on the anchor side: from "positive" it computes the expected domain $\max(\ell, 1)$ and compares *that* against the IR's declared domain. The string route loses the type; the predicate route does not.

Remark 1 (Type erasure is invisible to round-tripping)**.** Whenever a reduction is a non-injective quotient — it forgets a distinction that the surface language makes — a round-trip check that compares against the quotient's own rendering is structurally blind to exactly the forgotten distinction. This is the general form of A-lex's P2 failure, and it means the round-trip's blind spot is determined by the IR's *expressive gaps*, not by its accuracy.

3.6 A reliability checklist

The framework yields an operational checklist. It is short on purpose: every item is a question with a yes/no answer, and a "no" is a specific, actionable defect.

C1 Is there an explicit intermediate object at all, or does the model's prose answer go straight to a human?

C2 Is the verifier's domain of competence stated — which fragment, which logic, decidable or not?

C3 Is the reduction *monitored*, or only the final answer?

C4 Does at least one check compare against the *source statement*, rather than against a re-rendering of the intermediate object? (A "no" here means the pipeline is exposed to Remark 1.)

C5 Are unreadable or unconfirmed anchors treated as failures (abstain) or as passes (best effort), and is the chosen policy stated?

C6 Is vacuity distinguished from truth in the output?

C7 Are coverage, certification rate, and accuracy reported *separately*? A pipeline can be accurate, certifying, and useless.

C8 Is the reduction failure taxonomy instantiated with held-out examples, as in §3.3, and tested?

4 Experimental Design

4.1 Controlled language and corpus

We work in a controlled natural language whose semantic content is a record of four feature families: *quantifier*, *adjective* (the domain kind), *strictness* (the comparison), and *qualifier* (an optional congruence clause). Each family admits several synonymous surface realisations, so the same meaning can be written in many ways. The reductor translates a feature record to the IR of Definition 2 and may be *blind* to any subset of the families — a blind reductor silently normalises away a distinction it does not model, exactly as a language model does when it paraphrases a problem into its own reading.

Two modelling decisions make the measurements interpretable, and we state them because conflating them is the easiest way to produce a misleading result.

The reductor and the anchor extractor have different defects. The reductor is given *full* linguistic competence (it reads every synonym) but is blind to all four feature families: its defect is semantic, not lexical. The anchor extractor is a symbolic component with a *declared, closed* grammar: its defect is coverage, not semantics. Keeping these separate means a lexicon gap and a semantic blindness cannot be confused in the tables.

The corpus is split so that detection and cost are measurable. The *critical* set contains base–perturbed pairs, one pair per perturbation in each class, admitted only after checking that the ground truth actually flips; a perturbation that does not flip the truth value is discarded at generation time. The *benign* set contains strictly meaning-preserving paraphrases drawn from a population on which the blind reductor is provably faithful; here no answer is wrong, so every abstention is a pure certification cost. Without this split, a reductor blind to four families at once leaves almost no faithful instances against which to measure the price. We also report a *prevalence-controlled* corpus (every base instance plus 20% of the perturbed ones) because the balanced critical set inflates absolute error rates by construction.

4.2 Pipelines

We compare seven rungs, chosen so that each step changes exactly one thing. Their names in the tables are given verbatim in parentheses.

- gen_only — layer 1 alone: the model's reading, no IR, no certificate.
- verify — reduce, then decide the IR exactly. Sound and complete *on the IR*.
- verify+lexical_anchors — blind reduction, then an A-lex check; abstain unless every anchor is readable and holds.
- verify+typed_anchors — blind reduction, then an A-type check; same conservative policy.
- typed_anchored — *faithful* reduction, then an A-type check; abstain unless certifiable.
- typed_best_effort — faithful reduction; answer always, certify only when the anchors confirm.
- oracle — faithful reduction, perfect anchors: the ceiling.

The faithful reduction in the last three rungs is a *target*, not a claim that a real system attains it. Its role is to decompose the problem: the first three rungs isolate what anchors buy under an unfaithful reduction, and the last three isolate what is left once the reduction itself is trustworthy. A real system approximates the latter by restricting the accepted input to a language it can translate faithfully and abstaining outside it — which is exactly how formal and controlled-language front ends behave.

4.3 Metrics

We report four quantities per pipeline. *Coverage* is the fraction of instances answered. *Accuracy | answered* conditions on having answered. *Silent error rate* is the fraction of *all* instances on which a wrong answer was emitted *as if certified* — the quantity that Proposition 1 says can coexist with a perfect certificate rate. *Certificate rate* is the fraction carrying a machine-checkable object.

For the anchors themselves we report three quantities, because "detected" and "refused to answer" are not the same thing. An instance is *unfaithful* when the blind reductor's IR differs from the reference IR built from the sentence's true features — a property of the reduction, not of the answer. *Recall* counts violations: the anchor *contradicts* the IR, which is genuine detection. *Safety* counts unfaithful reductions whose answer was withheld for any reason. *Over-abstention* is the price: the share of faithful reductions refused certification.

5 Results

5.1 The blind spot is real

Across all 192 critical pairs, the blind reductor produced an IR *identical* for both members in 100% of cases, and the verifier returned an *identical* verdict in 100% of cases. This is Proposition 2 observed rather than assumed: the two sentences differ in truth value, and the entire pipeline, up to and including its certificate, cannot tell them apart. Any claim to the contrary would have to come from information that never entered the IR.

5.2 What verification alone buys, and what anchors buy

Table 1 shows the balanced critical set, where half the instances are by construction a semantics-changing edit; Table 2 is the headline. The second row is the one to look at. verify answers every question, is correct on 44.1% of them, and attaches a valid certificate to 100% of its answers. Its silent error rate is 55.9%: more than half of all queries receive a machine-checked wrong answer.

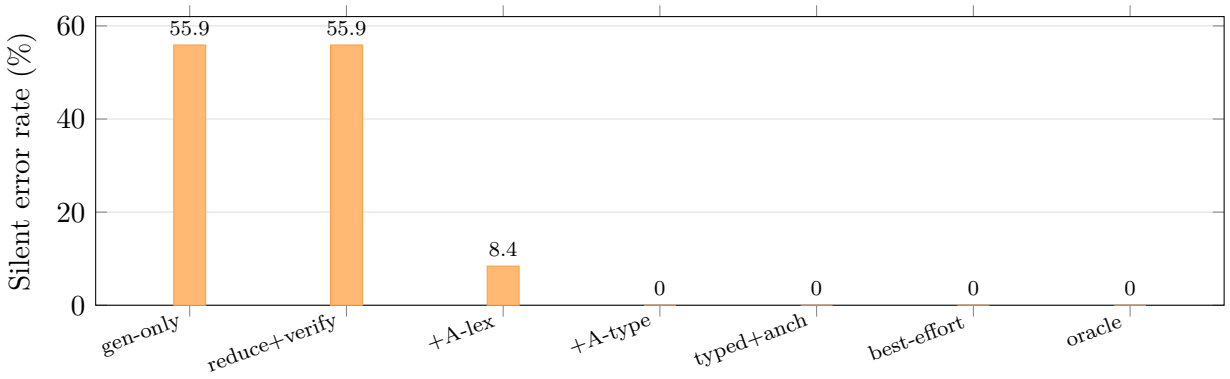


Figure 1: Silent error rate — wrong answers emitted as certified — on the prevalence-controlled corpus ($n = 238$). Reduction plus exact verification (verify) is the worst of both worlds: it certifies every answer and inherits every reduction error. Adding anchors removes the certified wrongness; making the reduction faithful removes the error itself.

Table 1: Behaviour on semantics-changing tasks (base + perturbed members of every critical pair). Coverage is the fraction answered; silent error rate counts wrong answers that were emitted as if certified; certificate rate is the fraction carrying a machine-checkable object.

Pipeline	n	Coverage	Acc. | ans.	Silent err.	Cert. rate
gen_only	384	100.0	50.0	50.0	0.0
verify	384	100.0	50.0	50.0	100.0
verify+lexical_anchors	384	26.6	80.4	5.2	26.6
verify+typed_anchors	384	19.8	100.0	0.0	19.8
typed_anchored	384	50.5	100.0	0.0	50.5
typed_best_effort	384	97.9	100.0	0.0	50.5
oracle	384	100.0	100.0	0.0	100.0

This is not a bug in the verifier — the verifier is sound and complete. It is Proposition 1 operating at scale.

The anchors move the certified wrongness. verify+lexical_anchors cuts silent error to 8.4% and verify+typed_anchors to 0%, at the cost of coverage: under the conservative "abstain unless every anchor is readable and holds" policy, the blind-reduction rungs can only certify a fifth or a tenth of the corpus. That is the honest trade: anchors do not make an unfaithful reduction correct, they make it *refuse to certify*. Reading the two levers together, the practical operating points are verify+typed_anchors+ (typed_anchored) for a conservative system that certifies about half of what it sees with no silent error, and typed_best_effort for a best-effort system that answers almost everything (96.6% coverage) with no silent error while honestly marking only 48.7% as certified.

5.3 Detection, safety, and the price of abstention

Table 3 separates detection from safety. IR verification alone has recall 0, as Corollary 1 requires. A-lex detects 91.8% of unfaithful reductions by contradiction but withholds only the same 91.8%; A-type detects slightly less (86.1%) but *withholds* 100% of them — it never certifies an unfaithful reduction, it merely fails to *contradict* a few of them, and those it abstains on. For a system whose failure mode is the certified wrong answer of Proposition 1, safety is the property that matters, and A-type buys it at a stated price: a higher over-abstention rate (48.2% against 39.1%).

Table 2: Behaviour on a prevalence-controlled corpus: every base instance plus 20% of the perturbed ones. Unlike Table 1 this is not balanced, so the absolute silent error rates are the ones a deployment of this shape would face.

Pipeline	n	Coverage	Acc. | ans.	Silent err.	Cert. rate
gen_only	238	100.0	44.1	55.9	0.0
verify	238	100.0	44.1	55.9	100.0
verify+lexical_anchors	238	21.4	60.8	8.4	21.4
verify+typed_anchors	238	11.8	100.0	0.0	11.8
typed_anchored	238	48.7	100.0	0.0	48.7
typed_best_effort	238	96.6	100.0	0.0	48.7
oracle	238	100.0	100.0	0.0	100.0

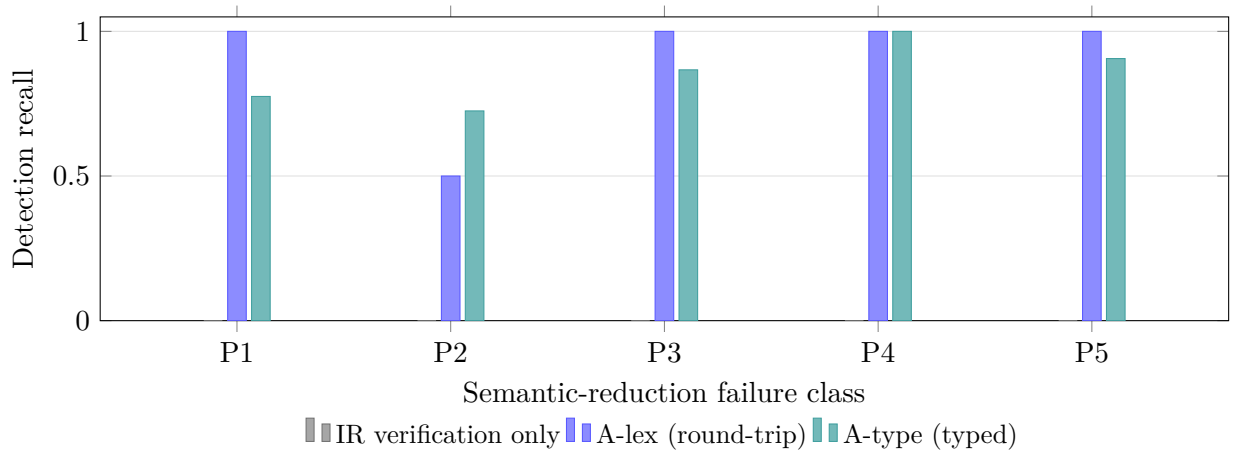


Figure 2: Detection recall by failure class. IR verification alone is zero everywhere by Proposition 2. A-lex is strong but non-monotone; A-type trades a little recall for uniform safety. The p=1.00 ablation in Table 4 shows that part of A-lex's apparent recall is paraphrase sensitivity rather than semantic detection.

The per-class rows expose the mechanism. On typing drift (P2), A-type recalls 72.5%; A-lex recalls 50% — and §5.4 shows that even that 50% is not what it appears to be.

5.4 The paraphrase confound, and why A-lex's recall is not what it looks like

The headline recall of A-lex is 91.8%, which invites the conclusion that a round-trip is a serviceable reduction-error detector. It is not, and the reason is visible only when the paraphrase factor is removed. We re-ran the detector analysis at $p = 1.00$, the setting in which every slot is realised with the exact form the anchor lexicon declares, so that no input falls outside the grammar and no synonym is present.

The result is unambiguous. With the paraphrase confound removed, A-lex's recall on P2 collapses to 0.0%, exactly as Remark 1 predicts: the IR encodes an interval, not a kind, so the round-trip regenerates the interval and the typing error leaves no trace to compare. At the same time A-type reaches 100% recall on *every* class, with zero over-abstention. The monotonicity is the tell. As p rises from 0.50 to 1.00, A-type's recall rises 73.4 → 86.1 → 100.0 and its over-abstention falls 80.7 → 45.7 → 0.0: better grammar coverage makes it strictly better. A-lex's recall is *non-monotone* — 96.3 → 91.8 → 83.6 — because at low p many of its "detections" are synonyms

Table 3: Mechanisms as detectors of *unfaithful reductions* – an IR that differs from the reference IR of the sentence, whether or not the answer happened to be right. *Recall* counts violations of an anchor (genuine detection); *safety* counts unfaithful reductions whose answer was withheld for any reason; *over-abstention* is the price, i.e. the share of faithful reductions refused certification. IR verification alone has recall 0 by Proposition 2.

Mechanism	Scope	$n_{\text{unf.}}$	Recall	Safety	$n_{\text{fai.}}$	Over-abst.
IR verification alone	all	244	0.0	0.0	220	0.0
A-lex (round-trip)	all	244	91.8	91.8	220	39.1
A-type (typed)	all	244	86.1	100.0	220	48.2
A-lex (round-trip)	P1	40	100.0	100.0	40	35.0
A-type (typed)	P1	40	77.5	100.0	40	45.0
A-lex (round-trip)	P2	40	50.0	50.0	40	50.0
A-type (typed)	P2	40	72.5	100.0	40	32.5
A-lex (round-trip)	P3	60	100.0	100.0	20	35.0
A-type (typed)	P3	60	86.7	100.0	20	50.0
A-lex (round-trip)	P4	40	100.0	100.0	40	42.5
A-type (typed)	P4	40	100.0	100.0	40	57.5
A-lex (round-trip)	P5	64	100.0	100.0	0	0.0
A-type (typed)	P5	64	90.6	100.0	0	0.0
A-lex (round-trip)	benign	0	0.0	0.0	80	35.0
A-type (typed)	benign	0	0.0	0.0	80	52.5

mistaken for semantic changes. A detector whose recall goes *down* when the input becomes more canonical is not detecting meaning; it is detecting wording. In the experiment's own vocabulary, part of A-lex's detection was over-abstention in disguise.

We state the practical consequence plainly: a round-trip check should not be reported as a reduction-error detector without the $p = 1.00$ control, because its headline recall conflates two very different signals — a real contradiction and a mere synonym — and only one of them is a reason to distrust the answer.

5.5 Vacuity: true, but about nothing

Table 5 covers the class that accuracy cannot see. All 32 claims are true only because their qualifying clause has no model at all: a universal statement over the empty set. gen_only and verify answer all 32 correctly and warn on none of them. They are not wrong — and they are not useful either: "true" reported without the caveat asserts a fact about objects that do not exist. typed_best_effort answers all 32, stays correct on all 32, and *warns on all* 32. This is the clearest illustration of why vacuity has to be a first-class outcome rather than a note: it is invisible to any metric that asks only whether the answer matched, and it is exactly where a pipeline most easily becomes confidently uninformative.

5.6 Sensitivity to the grammar gap

Anchor extractors that are not language models have a fixed grammar, and real input drifts off it. We therefore swept the paraphrase prior p across $\{0.50, 0.75, 1.00\}$ (Table 6). A-type's recall rises monotonically with p while its cost falls to zero, which is the behaviour of a detector whose only failure mode is coverage. A-lex's recall falls while its cost also falls, which is the signature of

Table 4: Detection at $p = 1.00$, i.e. every slot is realised with the exact form the anchor lexicon declares, so no input falls outside the grammar. This removes the paraphrase confound: A-lex's apparent detection of P2 vanishes entirely, showing that its headline recall was partly synonym sensitivity rather than semantic detection, whereas A-type's recall rises to 100% on every class it has a predicate for.

Class	$n_{\text{unf.}}$	A-lex rec.	A-type rec.	A-lex o.-a.	A-type o.-a.
P1	40	100.0	100.0	0.0	0.0
P2	40	0.0	100.0	0.0	0.0
P3	60	100.0	100.0	0.0	0.0
P4	40	100.0	100.0	0.0	0.0
P5	64	100.0	100.0	0.0	0.0

Table 5: Behaviour on the 32 claims that are true *only vacuously* (a universal statement whose qualifying clause has no model in the declared box). All of them are decided correctly by the rungs that answer them, but only the rungs that record vacuity as a first-class outcome actually *warn*. A pipeline that reports "true" without the warning has reported a fact that is true of no object.

Pipeline	n	Answered	Vacuity warned	Correct
gen_only	32	32	0	32
verify	32	32	0	32
verify+lexical_anchors	32	0	0	0
verify+typed_anchors	32	0	0	0
typed_anchored	32	13	13	13
typed_best_effort	32	32	32	32
oracle	32	32	32	32

a detector whose failure modes include *false alarms*. The practical reading: investing in grammar coverage pays off reliably for A-type and does not, by itself, fix A-lex.

5.7 The benign set: what abstention costs when nothing is wrong

Table 7 completes the picture from the other side. On the benign paraphrase set the blind reductor is faithful by construction, so no answer is wrong and *every* abstention is pure cost. The conservative anchored rungs give up roughly a third to a half of their coverage here (65.0% and 47.5%) for no gain in correctness, which is exactly the price the over-abstention column of Table 3 was measuring. The best-effort rung keeps 100% coverage while still certifying only 47.5%: it answers everything and is honest about what it can vouch for. Reading Table 7 against Table 1 is the cleanest statement of the trade the framework offers — the same mechanism that removes 55.9% silent error also refuses to certify roughly half of the answers that were never in danger.

6 Discussion

6.1 Which operating point to choose

The results suggest three regimes, and the right choice depends on what an error costs. Where a certified wrong answer is unacceptable and throughput is secondary, use a faithful reduction with typed anchors and abstain otherwise (typed_anchored): about half the corpus answered, nothing

Table 6: Sensitivity to how far the input distribution strays from the anchor grammar. p is the probability that a slot is realised with the form the lexicon was written for, so lower p means more input falls outside the declared grammar. Recall is detection by contradiction; over-abstention is the certification cost.

p	A-lex recall	A-lex over-abst.	A-type recall	A-type over-abst.
0.50	96.3	74.3	73.4	80.7
0.75	91.8	41.4	86.1	45.7
1.00	83.6	0.0	100.0	0.0

Table 7: Behaviour on the benign paraphrase set, where the blind reductor is provably faithful. Ground truth is unchanged throughout and no answer here is wrong, so every abstention is a pure certification cost.

Pipeline	n	Coverage	Acc. | ans.	Silent err.	Cert. rate
gen_only	80	100.0	100.0	0.0	0.0
verify	80	100.0	100.0	0.0	100.0
verify+lexical_anchors	80	65.0	100.0	0.0	65.0
verify+typed_anchors	80	47.5	100.0	0.0	47.5
typed_anchored	80	47.5	100.0	0.0	47.5
typed_best_effort	80	100.0	100.0	0.0	47.5
oracle	80	100.0	100.0	0.0	100.0

certified that is wrong. Where coverage matters and an uncertified answer can be routed to a human, use best effort (typed_best_effort): almost everything answered, nothing certified that is wrong, and the uncertified residue clearly marked. Where neither condition holds, plain verification should not be deployed alone at all — the second row of Table 2 is what it produces.

6.2 When anchoring cannot help

Anchoring is not free of assumptions, and the central one is worth stating loudly: **the anchor extractor must be independent of the reductor.** If the same model that produces the reading also produces the anchors, the anchors inherit the reading's blind spots, and by Proposition 2 they detect nothing. Our experiment models independence structurally — the reductor has full lexical competence and semantic blindness; the anchor extractor has a closed grammar and no semantic blindness — precisely because shared failure modes are the default in a real system. In deployment, independence means deriving anchors from somewhere other than the model's own paraphrase: the original problem statement, the formal statement attached to the dataset item, a textbook, or a differently-architected extractor. An anchored pipeline that shares one model between both roles has the appearance of a check and none of the substance.

Three further boundaries. First, our fragment is finite-domain affine arithmetic; real mathematics is not, and the exactness that makes our results trustworthy would be replaced by a sound-but-incomplete verifier, which weakens but does not remove the reduction problem — it merely moves the incompleteness inward. Second, anchors catch the failures their predicates encode, and no others: a reduction error outside the anchor language is invisible, which is why over-abstention and coverage must be reported. Third, our corpus is synthetic and deliberately adversarial; it measures mechanisms under known ground truth, not rates on real mathematical text. The mechanism claims (Propositions 1 and 2 and Remark 1) are language-independent, but the *magnitudes* should

not be read as field estimates.

7 Reproducibility and Verification of AI-Generated Claims

Every number, table, and figure in this paper is produced by a single deterministic driver, `run_experiment.py` (in the `experiments/` directory), from a fixed seed; re-running it regenerates the LaTeX fragments that are `\input` here, so the paper cannot silently drift from its evidence. The intermediate representation uses exact rational arithmetic throughout, so no verdict depends on a floating-point tolerance.

The claims in this paper were developed with heavy use of an AI assistant, and every AI-generated conclusion was checked against a primary source or an executed experiment before it was allowed into the text. Specifically, all 13 bibliographic entries were verified against the publisher record or the arXiv abstract page (venue, volume, pages, identifier) before being written into `references.bib` — none was accepted from model memory. The collaboration log (`AI_LOG.md`) records each prompt, what the model produced, what a human checked, and the outcome, including two cases in which the model's suggestion was wrong and was caught (a bibliographic year, and a statistical claim about the control set that the experiment contradicted). The seven-dimension after-action review (`AAR.md`) records the same process at the level of decisions rather than turns.

8 Conclusion

Reliability in AI for mathematics is usually pursued by strengthening the last layer of the pipeline: verify more, verify formally, verify reproducibly. We have argued, and then demonstrated on an exactly decidable harness, that this targets the wrong layer when the translation from prose to the verifier's object is left unmonitored. A sound verifier is sound only relative to the object it is handed, so an unfaithful reduction converts verification from a safety net into a certificate of wrongness; and because an IR-only verifier is a function of the IR, no amount of cleverness at that layer can detect a reduction that collapses two different meanings. The fix is not a stronger verifier but an independent source of meaning: anchors that do not pass through the reduction.

Not all anchors are equal. Comparing against a re-rendering of the intermediate object is attractive and, as we showed, partly illusory: it is structurally blind to exactly the distinctions the representation erases — notably typing — and its headline detection rate can be inflated by ordinary paraphrase. Asserting anchors as predicates over the object, by contrast, reaches full detection on every failure class we tested once the paraphrase factor is removed, and its only failure mode is coverage, which is measurable and improvable. The practical guidance is small and, we hope, durable: monitor the reduction, not only the answer; compare against the source, not only against a round trip; treat vacuity as an outcome rather than a footnote; and report silent error and certification rate separately, because a pipeline that certifies everything has certified nothing.

References

François Charton, Jordan S. Ellenberg, Adam Zsolt Wagner, and Geordie Williamson. PatternBoost: Constructions in mathematics with a little help from AI. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2411.00566.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28: 28th Interna-*

tional Conference on Automated Deduction, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635, Cham, 2021. Springer International Publishing. doi: 10.1007/978-3-030-79876-5_37.

Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2025. doi: 10.1038/s41586-025-09833-y.

Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. doi: 10.48550/arXiv.2210.12283.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. doi: 10.48550/arXiv.2305.20050.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.

Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint*, 2022. doi: 10.48550/arXiv.2211.14275.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, pages 32353–32368, 2022.

Kaiyu Yang, Gabriel Poesia, Jingxuan He, Wenda Li, Kristin Lauter, Swarat Chaudhuri, and Dawn Song. Formal mathematical reasoning: A new frontier in AI. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2412.16075.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. MiniF2F: a cross-system benchmark for formal olympiad-level mathematics. In *The Tenth International Conference on Learning Representations (ICLR)*, 2022. doi: 10.48550/arXiv.2109.00110.